



ALU instructions

R. Ferrero

Politecnico di Torino

Dipartimento di Automatica e Informatica (DAUIN)

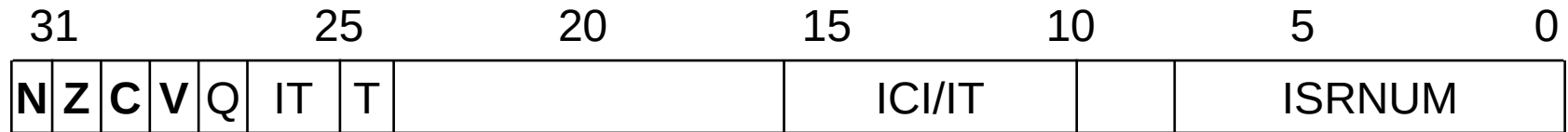
Torino - Italy

This work is licensed under the Creative Commons (CC BY-SA) License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>



Flags

- The uppermost nibble of the program status register contains the flags.



- Flags are set and cleared by means of:
 - comparison instructions, e.g., `CMP` and `TST`
 - ALU instructions if they have the suffix 'S'
 - a direct write to the program status register.

Negative flag N

- It corresponds to the first bit of the result.
 - if $N = 1$, a 2's complement number is negative.
- Example: $-4 + (-3) = -7 \rightarrow N = 1$

$$\begin{array}{r} 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100\ + \\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101\ = \\ \hline 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1001 \end{array}$$

- Why $N = 1$ in the following sum?

$$\begin{array}{r} 0111\ 0001\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ + \\ 0011\ 0101\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ = \\ \hline 1010\ 0110\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 \end{array}$$

Overflow flag V

- $V = 1$ if either one of these conditions occur:
 - the carry bit goes into the most significant bit
 - the carry bit comes out of the most significant bit

$$\begin{array}{r} 1010\ 0001\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 + \\ 1011\ 0101\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 = \\ \hline C = V = 1\quad 0101\ 0110\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 \end{array}$$

- $V = 0$ if both conditions occur: result is correct

$$\begin{array}{r} 1110\ 0001\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 + \\ 1011\ 0101\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 = \\ \hline C = 1, V = 0\quad 1001\ 0110\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 \end{array}$$

Carry flag C

- After a sum, $C = 1$ if the result size is 33 bit.

$$\begin{array}{r} 1010\ 0001\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ + \\ 1011\ 0101\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ = \end{array}$$

$$C = 1\ 0101\ 0110\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$$

- After a subtraction, C is inverted. Here $C = 1$:

$$\begin{array}{r} 0111\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ - \\ 0101\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ = \end{array}$$

$$0010\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$$

- $C = 1$ if minuend $\geq$ subtrahend (unsigned number)
- After a move or logical instruction, C is the result of an inline barrel shifter operation.

Zero flag Z

- It is set if the result is zero.

$$\begin{array}{r} 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101\ + \\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0011\ = \\ \hline 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 \end{array}$$

Comparison instructions

- They compare the value or test some bits:
`compare/test <Rd>, <operand2>`
- They set the flags without updating `Rd`.
- `operand2` can be:
 - a register with an optional shift
 - a constant obtained by shifting left an 8-bit value
 - a constant of the form `0x00XY00XY`
 - a constant of the form `0xXY00XY00`
 - a constant of the form `0xXYXYXYXY`.

CMP and CMN

- CMP (compare) subtracts `operand2` from `Rd` and updates the flags.
- CMN (compare negative) adds `operand2` from `Rd` and updates the flags.
- Examples with `r0 = #12`
 - `CMP r0, #10` -> `N = 0, Z = 0, C = 1, V = 0`
 - `CMP r0, #12` -> `N = 0, Z = 1, C = 1, V = 0`
 - `CMP r0, #14` -> `N = 1, Z = 0, C = 0, V = 0`
- The assembler can change CMP and CMN:
 - `CMP r0, #-8` becomes `CMN r0, #8`

TST and TEQ

- TST (test) computes the logical AND between `operand2` and `Rd`; then updates all the flags except `V`.
- TEQ (test equivalence) computes the logical EOR between `operand2` and `Rd`; then updates all the flags except `V`.
- Examples with `r0 = #1`
 - TST `r1, r0, LSL #4` -> is the 4th bit of `r1` set?
 - TEQ `r2, r3` -> are `r2` and `r3` equal?

Accessing Program Status Register

- `MRS <Rn>, <Sreg>` copies a special register into a register.
- `MSR <Sreg>, <Rn>` copies a general purpose register into a special register.
- `Sreg` can be APSR, EPSR, IPSR, and PSR.
- Example: `MRS r0, APSR` reads the flags and copies them to the uppermost nibble of `r0`.
- Example: `MSR APSR, r0` writes the flags using the uppermost nibble of `r0`.

ALU instructions and flags

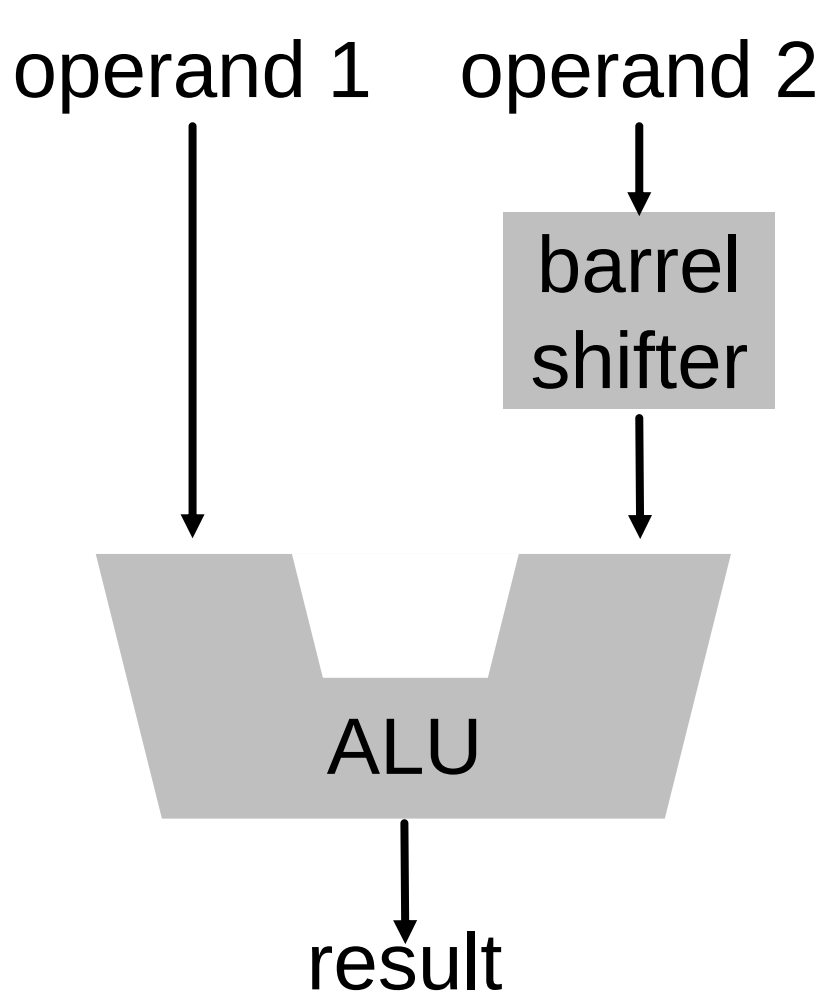
- ALU can perform:
 - arithmetic operations
 - logic operations
 - shift and rotate operations.
- The flags are affected only if the suffix 'S' is appended to the instruction.

- `LDR r0, =0xFFFFFFFF9`

`ADDS r1, r0, #7`

Flags are $N = 0$, $Z = 1$, $C = 1$, $V = 0$.

Internal data path to ALU



Operand 1 is a register.

Operand 2 can be:

- shifted register
- shifted 8-bit constant
- 0x00XY00XY
- 0xXY00XY00
- 0xXYXYXYXY
- 12-bit constant (`ADD` and `SUB` only).

Arithmetic instructions

- Addition
 - addition with carry
- Subtraction
 - subtraction with carry
 - reverse subtraction
- Multiplication
 - multiplication with accumulation
 - multiplication with subtraction
- Division

Addition

- `ADD <Rd>, <Rn>, <op2>`

$$Rd = Rn + op2$$

- `ADC <Rd>, <Rn>, <op2>`

$$Rd = Rn + op2 + C$$

- `ADDW` is like `ADD`, but it takes only a 12-bit value and it can not update flags.
- With `ADC` it is possible to add 64-bit values:

`ADDS r4, r0, r2`

`ADC r5, r1, r3`

In the example `r5, r4 = r1, r0 + r3, r2`

Subtraction

- SUB <Rd>, <Rn>, <op2>

$$Rd = Rn - op2$$

- SBC <Rd>, <Rn>, <op2>

$$Rd = Rn - op2 + C - 1$$

- SUBW is like SUB, but it takes only a 12-bit value and it can not update flags.
- With SBC, 64-bit values can be subtracted

SUBS r4, r0, r2

SBC r5, r1, r3

In the example $r5, r4 = r1, r0 - r3, r2$

Reverse subtraction

- RSB <Rd>, <Rn>, <op2>

$$Rd = op2 - Rn$$

- Advantages:

- either one or the other operand can be shifted before the subtraction

```
;r0 = r1 - r2*4
```

```
SUB r0, r1, r2, LSL #2
```

```
;r0 = r1*4 - r2
```

```
RSB r0, r2, r1, LSL #2
```

- a register can be subtracted from a constant.

Multiplication

- multiplication with 32-bit result

`MUL <Rd>, <Rn>, <Rm>`

- There is no distinction between signed and unsigned multiplication with 32-bit result.

- unsigned multiplication with 64-bit result

`UMULL <RLo>, <RHHi>, <Rn>, <Rm>`

- signed multiplication with 64-bit result

`SMULL <RLo>, <RHHi>, <Rn>, <Rm>`

- All operands must be registers.
- Suffix 'S' can not be added (Cortex-M3 only)

Multiplication with accumulation

- MLA $\langle Rd \rangle, \langle Rn \rangle, \langle Rm \rangle, \langle Ra \rangle$

$$Rd = Rn * Rm + Ra$$

- MLS $\langle Rd \rangle, \langle Rn \rangle, \langle Rm \rangle, \langle Ra \rangle$

$$Rd = Rn * Rm - Ra$$

- UMLAL $\langle RLo \rangle, \langle RHHi \rangle, \langle Rn \rangle, \langle Rm \rangle$

$$RHHi, RLo = Rn * Rm + RHHi, RLo$$

- SMLAL $\langle RLo \rangle, \langle RHHi \rangle, \langle Rn \rangle, \langle Rm \rangle$

same as UMLAL, but with signed values.

Avoiding multiplication

- Thanks to the inline barrel shifter, some multiplications can be performed without using the multiplier array
 - multiplication by 2^n
`LSL r1, r0, #3 ; r1 = r0*8`
 - multiplication by $2^n + 1$
`ADD r1, r0, r0, LSL #3 ; r1 = r0*9`
 - multiplication by $2^n - 1$
`RSB r1, r0, r0, LSL #3 ; r1 = r0*7`
- It saves power and execution time.

Division

- unsigned division

UDIV <Rd>, <Rn>, <Rm>

- signed division

SDIV <Rd>, <Rn>, <Rm>

- $Rd = Rn / Rm$

- If Rn is not exactly divisible by Rm , the result is rounded toward zero.

- Suffix 'S' can not be added (ARM v7-M)

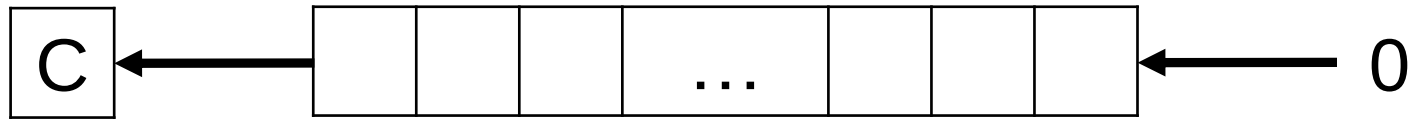
- UDIV and SDIV never change flags.

Logic instructions

- `AND <Rd>, <Rn>, op2 ; Rn AND op2`
- `BIC <Rd>, <Rn>, op2 ; Rn AND NOT op2`
- `ORR <Rd>, <Rn>, op2 ; Rn OR op2`
- `EOR <Rd>, <Rn>, op2 ; Rn XOR op2`
- `ORN <Rd>, <Rn>, op2 ; Rn OR NOT op2`
- `MVN <Rd>, <Rn> ; NOT Rn`

Shift instructions

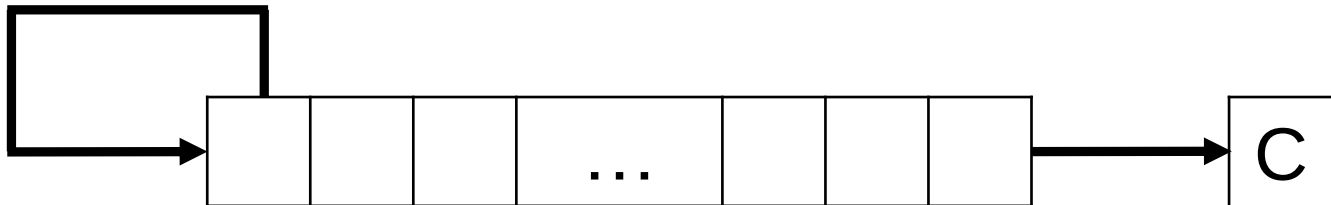
- LSL <Rd>, <Rn>, <op2>



- LSR <Rd>, <Rn>, <op2>

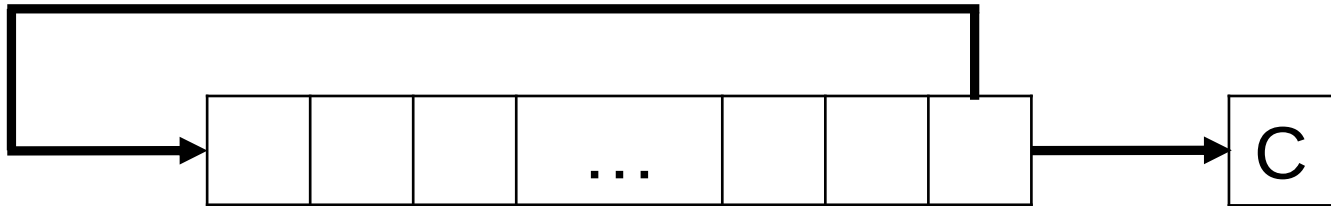


- ASR <Rd>, <Rn>, <op2>

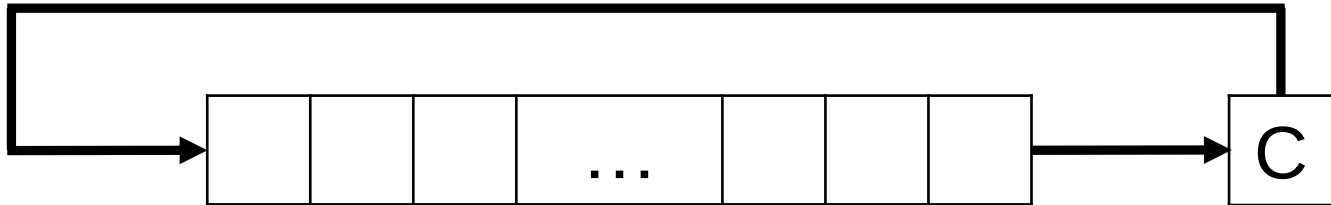


Rotate instructions

- ROR <Rd>, <Rn>, <op2>



- RRX <Rd>, <Rn>

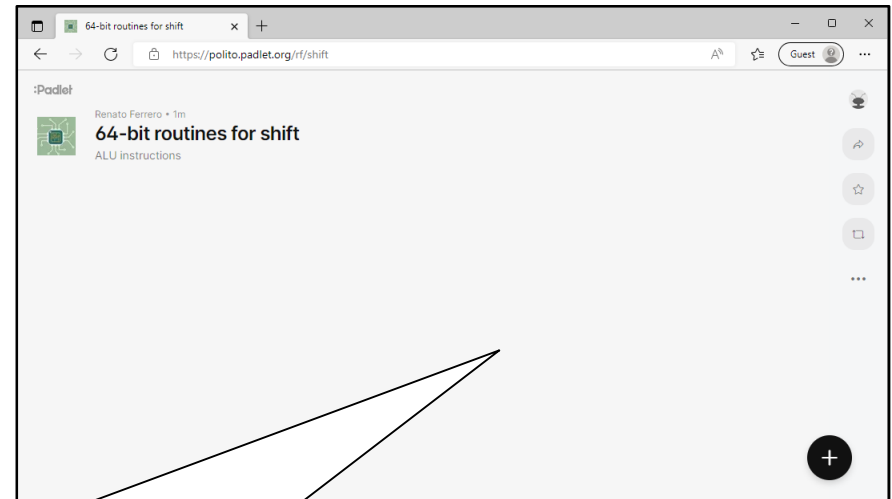


Exercises: 64-bit routines for shift

- 1) shift left $r1, r0$ by one bit and save the result in $r3, r2$.
- 2) logical shift right $r1, r0$ by one bit and save the result in $r5, r4$.
- 3) arithmetic shift right $r1, r0$ by one bit and save the result in $r7, r6$.
- 4) rotate right $r1, r0$ by one bit and save the result in $r9, r8$.
- 5) rotate right through carry flag $r1, r0$ by one bit and save the result in $r11, r10$.

Try using as few instructions (2 - 3) as possible.

<https://polito.padlet.org/rf/shift>



To write a post, click over  or double click the background